



Underworld's lightweight cloud for online classrooms.

Louis Moresi¹ 

¹Australian National University

DOI <https://doi.org/10.6084/m9.figshare.33193422>

Abstract

We built a cheap-and-cheerful solution with persistent storage and a binder-like access to notebooks in a repository that is aimed at serving a single classroom. The zero-to-server time is just a few minutes and there is minimal manual configuration.

Keywords AuScope UW Cloud, Underworld Code, Python/Jupyter

One of the most popular cloud services for python / jupyter - based codes is binder (www.mybinder.org) which can transform an online repository into a running instance in the cloud with little effort on the part of the owner of that repository to set it up and no effort at all for the end user. Access to binder has transformed classes with computational content by allowing flexible delivery of content that does not have to depend on a standardized software stack that has been approved months in advance by a specialist IT group.

In recent weeks lectures at schools and universities around the world have moved to on-line classrooms and one shortcoming of binder has become apparent — the lack of persistence for work that spans an extended work session. The fact that students (and lecturers) working at home may be forced to work in a fragmented fashion means that they often time-out access in binder and have to restart a new, pristine session each time they return.

We built a cheap-and-cheerful solution with persistent storage and a binder-like access to notebooks in a repository that is aimed at serving a single classroom. We provide a template github repository that can be used to configure, manage and monitor a single [digital-ocean](#) droplet (any server, really) that serves up content via* [the-littlest-jupyter-hub](#).* The zero-to-server time is just a few minutes and there is minimal manual configuration.

It goes something like this:

1. Clone the [github template repository](#)
2. Setup a new ubuntu droplet on digital ocean
3. Add the IP address, password and preferred admin user details to the repository SECRETS

4. Update the conda requirements (and, if necessary, apt packages) files
5. Commit the changes and wait for the server to come up
6. Log in !
7. *Optional: add default content and personalise the README for your repository.*

We use github actions in the repository to initialize, update and monitor the server. The workflow files have some optional configuration information in them that, if updated, will also trigger the server to rebuild / re-initialise.

The template repository itself runs a demonstration server at <https://demon.underworldcloud.org>. (If you want to use https you also need to configure a hostname for your server. Information can be found at [the-littlest-jupyter-hub](#) documentation pages.)

1. WHAT DO THE USERS SEE ?

Why not try it out and see for yourself ? Users first need to sign up to use the server. In our demo version, they can just request an account via a signup page:



The server is then accessed either via the hub url itself or via a link that also populates the notebooks etc in the style of a binder link.



We use [nbgitpuller](#) to draw in content for each user on the fly. nbgitpuller is designed to distribute content in a repository to students and to manage (gracefully) the issues associated with merging updated content and existing work. There is a link generator that can be used to make the badges for users to launch the server for a given repository.

For more information check out the template repository: <https://github.com/underworld-geodynamics-cloud/underworld-cloud-droplet> or contact [Louis Moresi](#)

2. ADMINISTRATION LAYER

The class server can be administered by an instructor who does not have to have access to the digital ocean console. Most everyday tasks can be managed via the jupyterhub console.



There is a page that can be used to authorize or un-authorize users' access to the service.



The admin users of the hub are also able to configure the server itself via the jupyter terminal.

3. WHY DIGITAL OCEAN ?

Digital ocean provides very flexible virtual servers (droplets) that can be spun up and down very quickly and very cost effectively. It is simple to resize a resource and the whole process can, in principle, be automated.